

졸업작품 제안서 Project P.A

역-공급망 경영 시뮬레이션 게임

202121026 컴퓨터공학과 신동준



| 기획 의도 및 핵심 정체성



역발상의 재미

기존 '심라이프'의 장르적 매너리즘을 타파하기 위해 플레이어를

'노동자'에서 '관리자'로 전환합니다.

노동은 NPC가, 시스템 최적화는 플레이어가 담당하는 '역-공급망' 구조를 핵심 재미로 삼습니다.



안락한 경영

힐링 게임이 주는 시각적·청각적 평온함 아래에 치밀하게 설계된 경제

시스템과 MBTI 기반의 Npc 로직을 배치함으로써, 감성적인 만족과

논리적인 성취감을 동시에 충족

친구의 섬에 '파견 직원'으로 입주하는 실질적인 멀티플레이어 협력을 지향합니다.

벤치마킹



Animal Crossing

- ✓ **강점:** 감성적인 비주얼, NPC와의 깊은 정서적 유대감 및 힐링 요소.
- ⚠ **한계:** 후반부 콘텐츠 부족 및 반복적인 수동 노동 (채집)의 피로도.



Stardew Valley

- ✓ **강점:** 방대한 테크 트리, 농사/전투/커뮤니티가 결합된 완성도 높은 경제.
- ⚠ **한계:** 1인 개발의 한계로 인한 후반부 자동화 시스템의 부재.

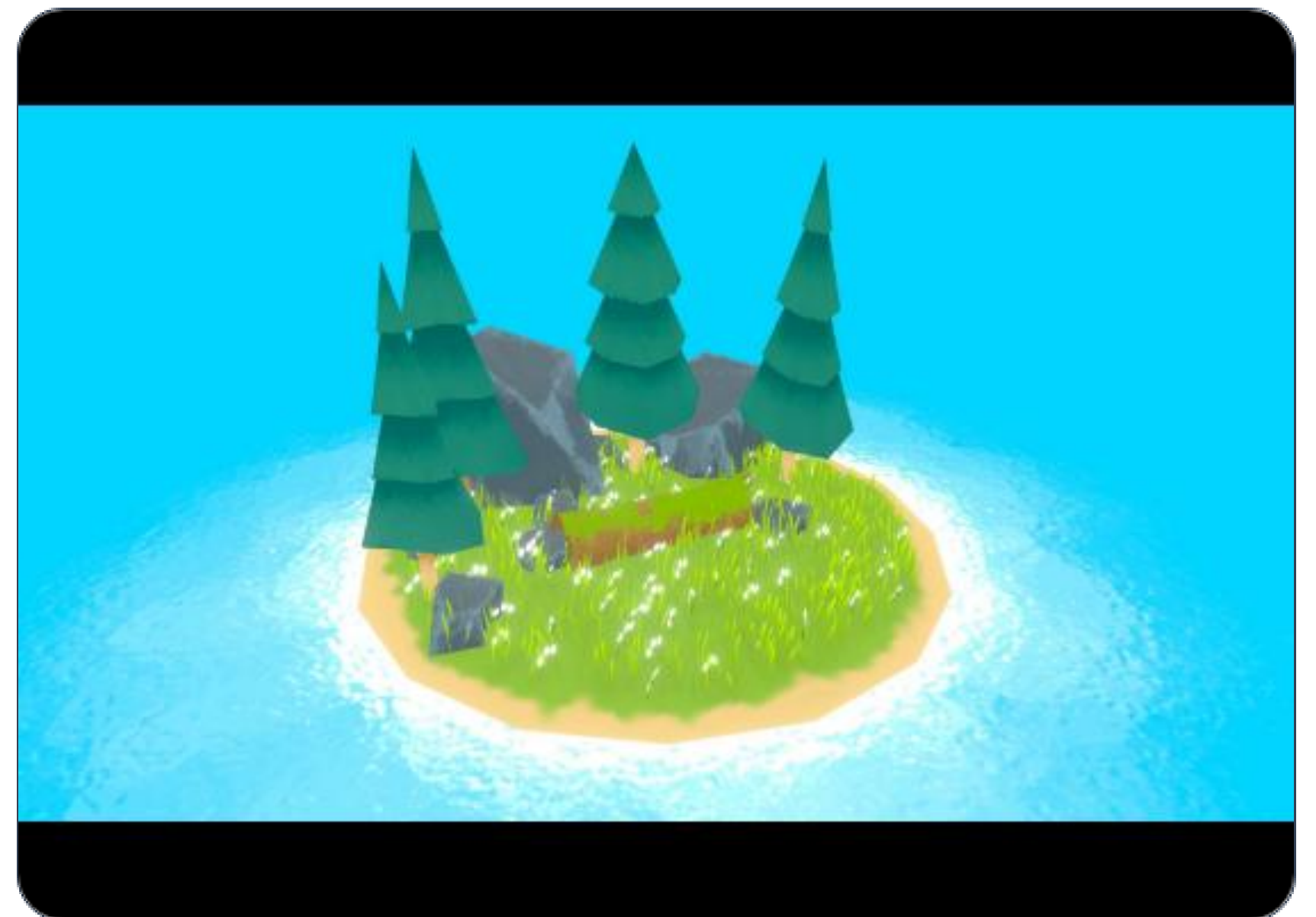


DAVE THE DIVER & Dinkum

- ✓ **강점:** 개성있는 개척 시스템 (딩컴)
강점: 유쾌한 연출 및 직원 고용 시스템으로 상점 운영의 재미 (데이브 더 다이버)
- ⚠ **한계:** 산업적인 분위기로 인한 감성 결여 및 높은 진입 장벽.

핵심 게임 메커니즘 : 경제 순환 및 주요 기능

1. 1차 생산: : 농부, 광부, 벌목꾼으로 명명된 NPC들은 매일 아침 각자의 일터로 이동하여 원목, 광석, 작물 등의 기초 자원을 채집.
2. 유통 및 매입: NPC는 수확한 결과물을 들고 사용자의 상점을 방문 -
> 사용자는 시세를 고려하여 이를 매입하며, 이 과정에서 지역 경제에 유동성을 공급.
3. 2차 가공: 매입한 원자재는 사용자의 공방이나 2차 전문가 NPC(쉐프, 디자이너 등)에게 의뢰되어 가공품으로 변환하여 아이템의 가치를 상승.
4. 최종 소비: 가공된 생필품과 사치품을 다시 판매하여 수익을 창출하고 마을 인프라 확장 및 기술 연구에 재투자.



| 본사 승인 등급 (Tier) 시스템

플레이어의 성장은 단순한 숙련도 상승이 아닌, '본사 승인 등급'의 해금에 따라 결정된다.
이는 자본과 신용에 기반한 권한 확대 시스템으로 작동한다.

티어	명칭	해금 조건	사용자 권한	NPC 행동 변화
Tier 1	지점장	초기 대출 상환	상점 건설, 기초 도구 판매	자원을 상점에 판매하기 시작
Tier 2	관리자	매출 10만 G	공방 건설, 철제 도구 해금	도구를 구매하여 특수 자원 채집
Tier 3	사업가	마을 평판 Lv.3	특화 건물 건설, 자동화 도입	가공품 납품 및 소비 활동 증가
Tier 4	파트너	본사 감사 통과	무역항 해금, 사치품 거래	가족/제자 영입을 통한 인구 증가

| 멀티플레이 시스템 설계

네트워크 아키텍처 -> 클라이언트 - 서버(Host-Client) 모델

사용자가 방장이 되어 자신의 섬을 서버로 개설하면, 친구는 '파견 직원' 신분으로 접속

세션 관리 -> Unity 6의 SDK를 사용하여 세션 생성, 참여 코드 발행, 플레이어 그룹화를 관리

호스트의 역할:



☐ 서버 권한: 게임의 핵심 로직을 통제.
NPC의 AI 행동 결정, 경제 시스템의 재화 계산, 상점의 재고 관리 등은 모두 호스트의 컴퓨터에서 계산.

☐ 데이터 소유: 섬의 원본 데이터를 로드하고 세션을 생성합니다.

클라이언트(파견 파트너)의 역할:



☐ 호스트가 생성한 세션 코드를 입력하여 접속합니다.

☐ 자신의 캐릭터 조작 정보만 서버로 보내고, 서버(호스트)로부터 업데이트된 월드 정보를 받아 화면에 송출.

연결의 핵심 (UGS Relay):



☐ 일반적인 가정용 인터넷은 보안(NAT) 때문에 외부 접속이 어렵기 때문에 Unity Relay는 호스트와 클라이언트 사이에서 중계소 역할을 하여 포트 포워딩 같은 복잡한 설정 없이도 전 세계 어디서든 연결되게 돕습니다.

데이터 동기화 방식

Unity 6의 Netcode for GameObjects(NGO) 라이브러리를 활용한 설계

환경구성

NetworkManager - 프로젝트의 중앙 통제실로, Unity Gaming Services(UGS)의 Relay와 연동하여 세션을 관리하고 플레이어의 접속을 처리

NetworkObject 컴포넌트 - 동기화가 필요한 모든 오브젝트(플레이어, 상점, NPC)에 이 컴포넌트를 부착하여 네트워크 상에서 고유 ID를 부여받도록 구현

데이터 유지

실시간 상태 동기화: NetworkVariable 구현

- 구현 방식: 데이터를 일반 변수가 아닌 `NetworkVariable<T>` 클래스로 감싸서 선언합니다.
- 권한 설정: 보안을 위해 오직 서버(호스트)만 이 값을 수정할 수 있도록 `NetworkVariableWritePermission.Server` 권한을 부여합니다.
- 이벤트 바인딩: 값이 변할 때마다 `OnValueChanged` 델리게이트를 통해 UI가 자동으로 갱신되도록 Observer 패턴을 적용

상호작용 처리

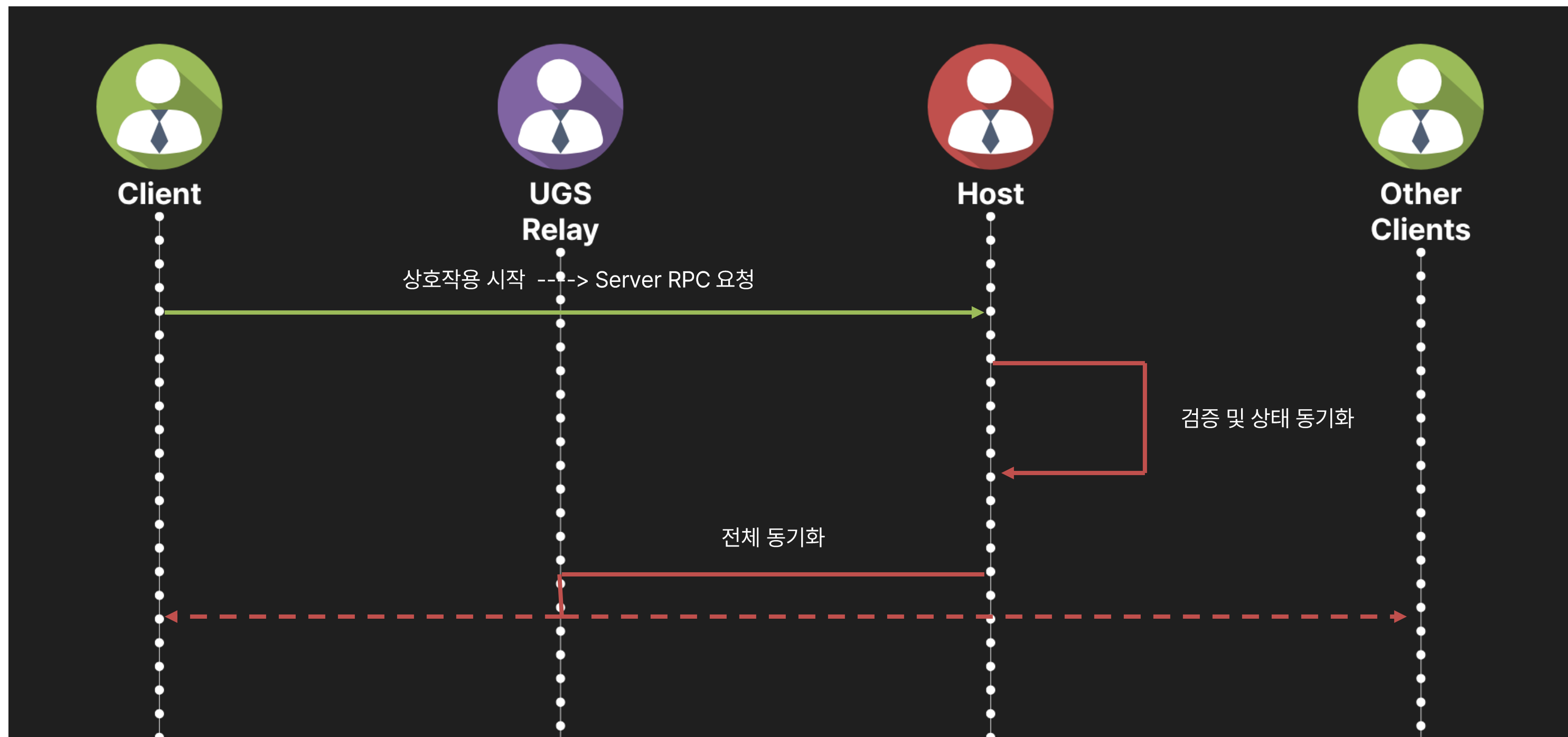
일회성 이벤트 처리: RPC 구현

- `ServerRPC` (클라이언트 → 서버): 클라이언트가 아이템을 구매하거나 자원을 채집할 때, 서버에게 "내 돈을 줄이고 인벤토리에 넣어줘"라고 요청하는 메서드에 `[ServerRpc]` 속성을 붙여 구현합니다.
- `ClientRPC` (서버 → 클라이언트): 서버가 요청을 승인한 후, 모든 플레이어에게 "A가 나무를 베으니 나무 쓰러지는 애니메이션을 보여줘"라고 명령할 때 `[ClientRpc]` 속성을 사용합니다.

멀티플레이 시퀀스 다이어그램

Client (Player): 상호작용을 시작하는 플레이어
UGS Relay: 데이터 전달을 중계하는 서버

Host (Server): 게임 로직을 검증하고 데이터를 소유하는 방장
Other Clients: 변경된 월드 상태를 전달받는 다른 유저들



| 모델링용 AI 탐색

MESHY.AI와 TRIPO3D.AI를 활용한 생성형 AI 파이프라인을 구축하여 에셋 생산 속도를 가속화 예정

비교 항목	TRIPO3D.AI (V3.1 Ultra)	MESHY.AI
핵심 강점	'Smart Mesh'를 통한 게임 최적화 토폴로지 생성	고품질 텍스처링 및 정밀한 메쉬 제어
토폴로지 품질	쿼드(Quad) 기반의 깔끔한 엣지 플로우	수동 리토폴로지가 필요한 경우가 있음
애니메이션	원클릭 오토 리깅(Rigging) 및 Mixamo 연동	100여 개의 애니메이션 프리셋 지원
생산 속도	약 5초 내 초기 생성 가능	약 45~60초 소요
가격 정책	\$12/월 (200 크레딧)	\$16/월 (200 크레딧)

아트 컨셉 및 비주얼 스타일 가이드

시각적 지향점 -> 로우 폴리(Soft Low-Poly)



Trip3d.ai에서 생성한 모델

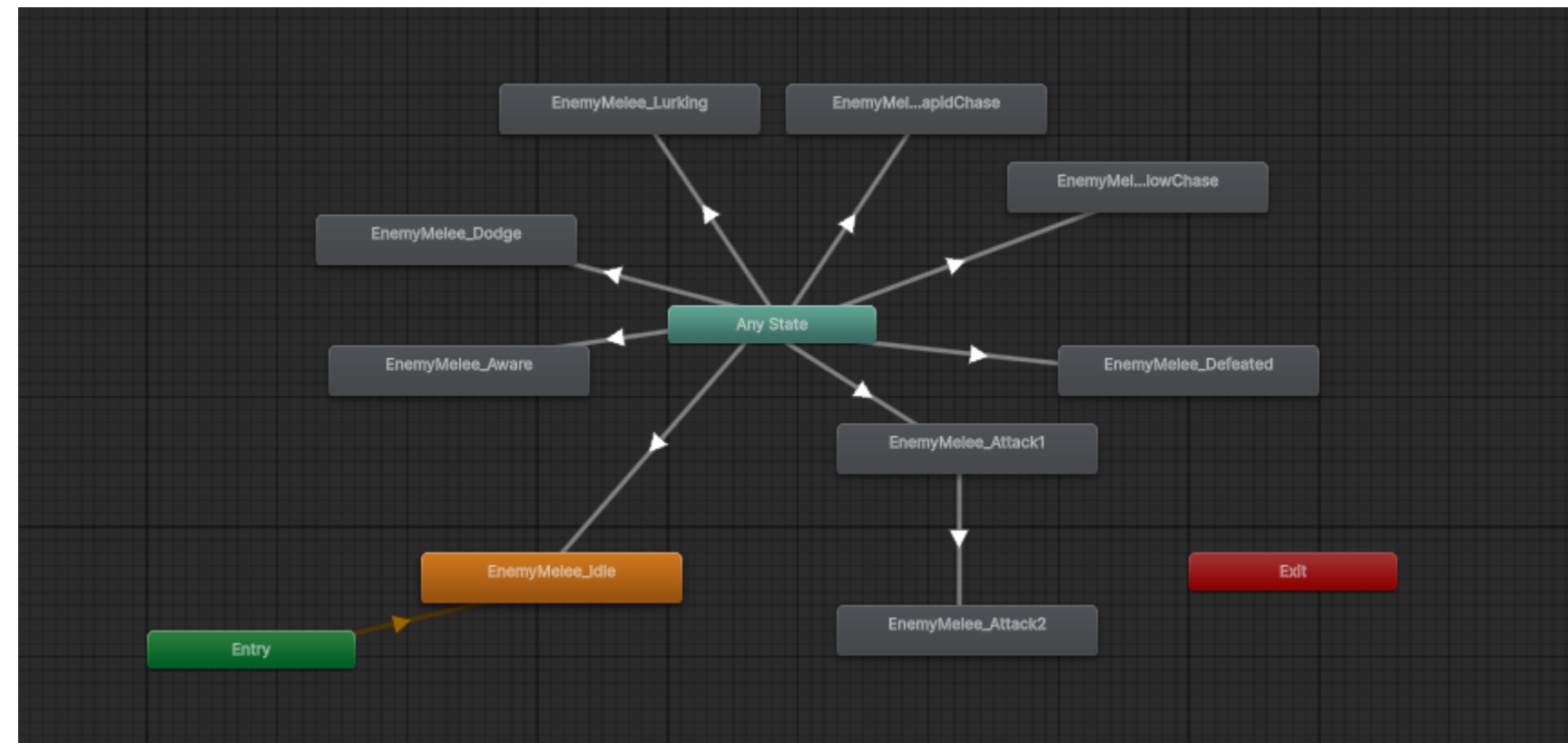
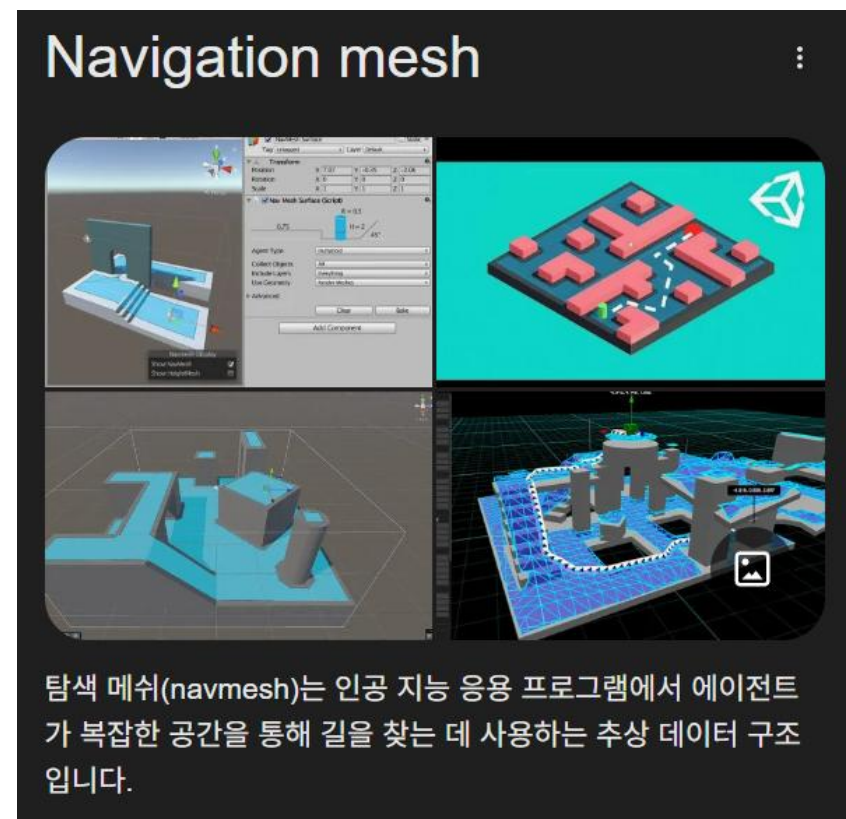
비주얼 스타일의 핵심 요소

- 부드러운 그라데이션 텍스처: 명확한 외곽선보다는 부드러운 색감 변화를 통해 따뜻한 분위기를 연출한다.
- URP 기반 Stylized Lighting: PBR(Physically Based Rendering) 재질을 사용하되, 라이팅 설정을 통해 실사보다는 만화에 가까운 느낌을 준다. Ambient Occlusion을 적극 활용하여 사물의 거리감을 강조한다.
- Curved World 효과: '동물의 숲'의 아이코닉한 특징인 지평선 굴곡 효과를 셰이더로 구현한다. 이는 실제 지형을 구 형태로 만드는 것이 아니라, 카메라와의 거리에 따라 정점(Vertex)을 Y축으로 오프셋 시키는 연산 방식을 통해 리소스를 절약한다.

MBTI 기반 NPC 행동 로직

MBTI 차원	AI 반영 변수 (Internal Variable)	실제 행동 양상
E (외향)	Social_Weight, Area_Radius	광장 체류 시간 증가, 주변 NPC 행복도 버프 제공
I (내향)	Work_Efficiency, Home_Stay_Time	집에서의 작업 시간 증가, 생산 효율 가산점
S (감각)	Utility_Consumption_Rate	도구, 식재료 등 실용적인 물건 위주의 구매 패턴
N (직관)	Luxury_Consumption_Rate	비싼 가구, 장식품 등 심미적 가치 중심의 소비
J (계획)	Schedule_Drift_Value	정해진 일과표(Work-Rest)를 엄격하게 준수
P (인식)	Random_Event_Probability	일과 외 돌발 행동이나 희귀 아이템 채집 변수

실제 구현: NavMesh & FSM



NPC의 상태는 FSM(Finite State Machine)으로 관리되어, 성향에 따라 전이 확률 조정

예를 들어, 휴식 상태에서 '광장으로 이동'할 확률은 E형 NPC가 I형보다 현저히 높게 설정된다.
모든 경로는 NavMesh를 통해 계산되어 마을의 지형지물과 충돌하지 않고 자연스럽게 이동

| 기술 스택 및 시스템 아키텍처



Engine & Language

Unity 6 (URP)
C# Scripting

최신 버전의 Unity 엔진을 통해 안정적인
게임을 구현합니다.



Architecture

Unity-style MVC
Observer Pattern

데이터(Model)와 로직(Controller), 화면
(View)을 분리하여 유지 보수성을
확보합니다.



Data & AI

ScriptableObject
JSON / NavMesh

유니티의 ScriptableObject 기반 정적 데이
터 관리 및 JSON 직렬화를 통한 저장 시스
템을 구축합니다.

시스템 아키텍처: MVC 구조

MODEL 구조

Model은 ScriptableObject와 JSON을 활용하여 정적 및 동적 데이터를 관리합니다. 이를 통해 게임 내 모든 데이터의 효율적인 저장 및 처리가 가능합니다.

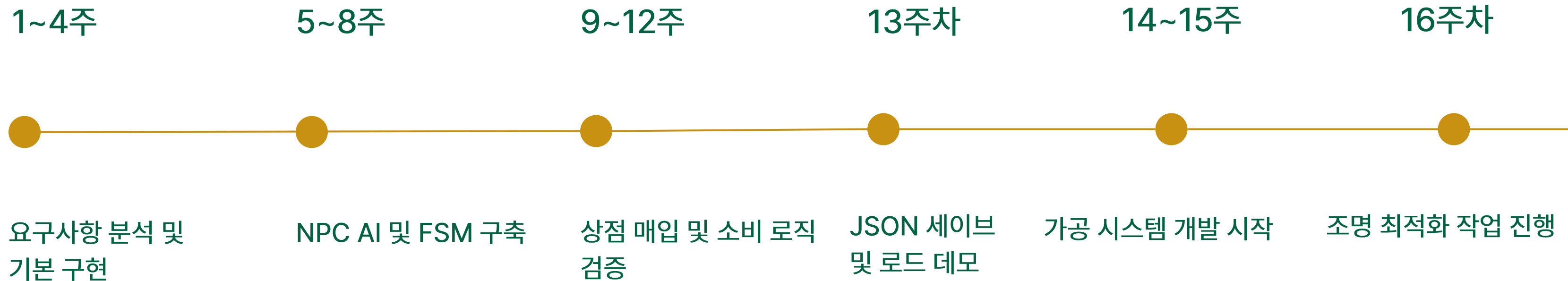
VIEW 렌더링

View는 URP 렌더링을 사용하여 고품질의 그래픽을 제공합니다. Observer 패턴을 통해 UI를 실시간으로 갱신하여 사용자 경험을 향상시킵니다.

CONTROLLER 역할

Controller는 InteractionManager, EconomySystem, NpcAIController로 구성되어 있습니다. 이들은 게임 내 상호작용과 경제 흐름을 최적화하여 플레이어 경험을 극대화합니다.

I 1학기 개발 로드맵



시나리오 및 세계관 설계

프로젝트 P.A.의 배경 -> 고립되었지만 자원이 풍부한 미개척 제도

플레이어의 역할 : 플레이어는 본사(P.A. Headquarters)에서 파견된 '개척 지원 지점장'으로서, 각기 다른 사연을 가진 NPC들과 함께 섬을 발전시켜야 한다.

시나리오 목표: 사용자는 초기 대출금을 상환하고, 최종적으로 섬을 본사가 인정하는 '최고 등급 파트너 섬'으로 승격시켜야 한다.

중간 감사 시스템: 특정 주기마다 본사에서 감사원이 파견되어 경제 지표(매출, NPC 행복도)를 체크합니다. 성적이 좋으면 인센티브(특수 에셋)를 주고, 나쁘면 경고를 부여.

NPC 대화 및 관계: MBTI 성향에 따라 NPC의 대사 톤과 관심사가 달라진다. 예를 들어, T형 NPC는 경제적 효율성을 논하는 대사를, F형 NPC는 섬의 분위기와 친밀도에 관한 대사를 주로 던진다.

I 기대효과 및 활용 방안

장르 혁신

Project P.A.는 Cozy Management 장르를 혁신하며, 다양한 플레이 스타일을 지원하여 새로운 시장을 창출할 것으로 기대합니다.

효율적 개발 모델 구축

AI 모델링 도구를 적극 활용하여 1인 개발자가 직면하는 리소스 부족 문제를 해결하고, 퀄리티 있는 결과물을 창출하는 표준 파이프라인을 입증한다.

크로스 플랫폼 시장성

PC의 정밀함과 모바일의 접근성을 결합하여 폭넓은 사용자 층을 확보할 수 있는 상업적 가능성을 확보한다.

피드백 반영

I 아이템 엔티티 설계

ItemData (ScriptableObject)
- ItemID (int): 고유 식별 번호 (PK) - ItemType (Enum): Raw(원목), Processed(판재), Furniture(의자) - BaseBuyPrice (int): NPC로부터 매입할 때의 기준가 - IdealSellPrice (int): 시스템 권장 판매가 - MBTISensitivity (float[]): [S, N, P, J] 각 성향에 대한 가격 저항선 가중치

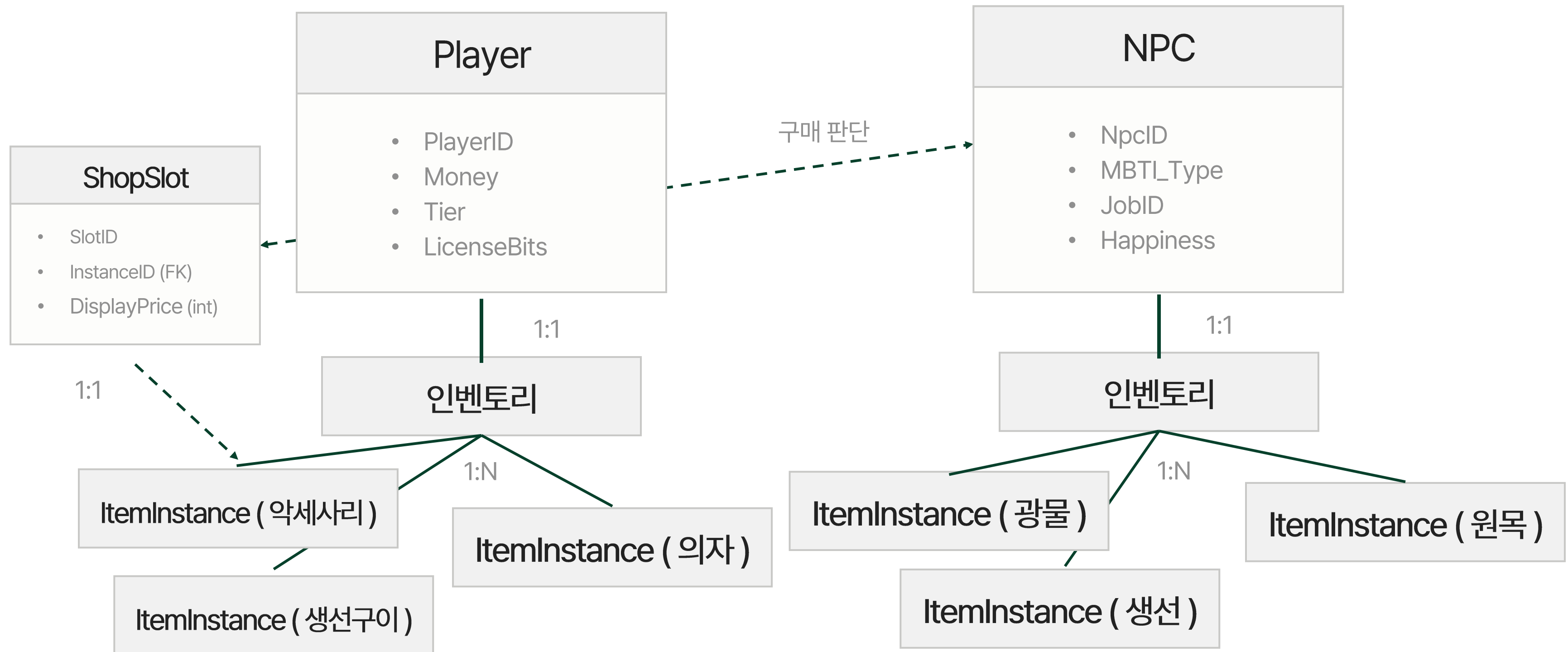
원본 데이터 참조
← - - - - -

ItemInstance (Class))
- InstanceID (Guid): 개별 아이템의 고유 식별자 - CurrentPrice (int): 유저가 직접 책정한 현재 가격 - Quality (float): 가공 품질 (가치 상승 계수)

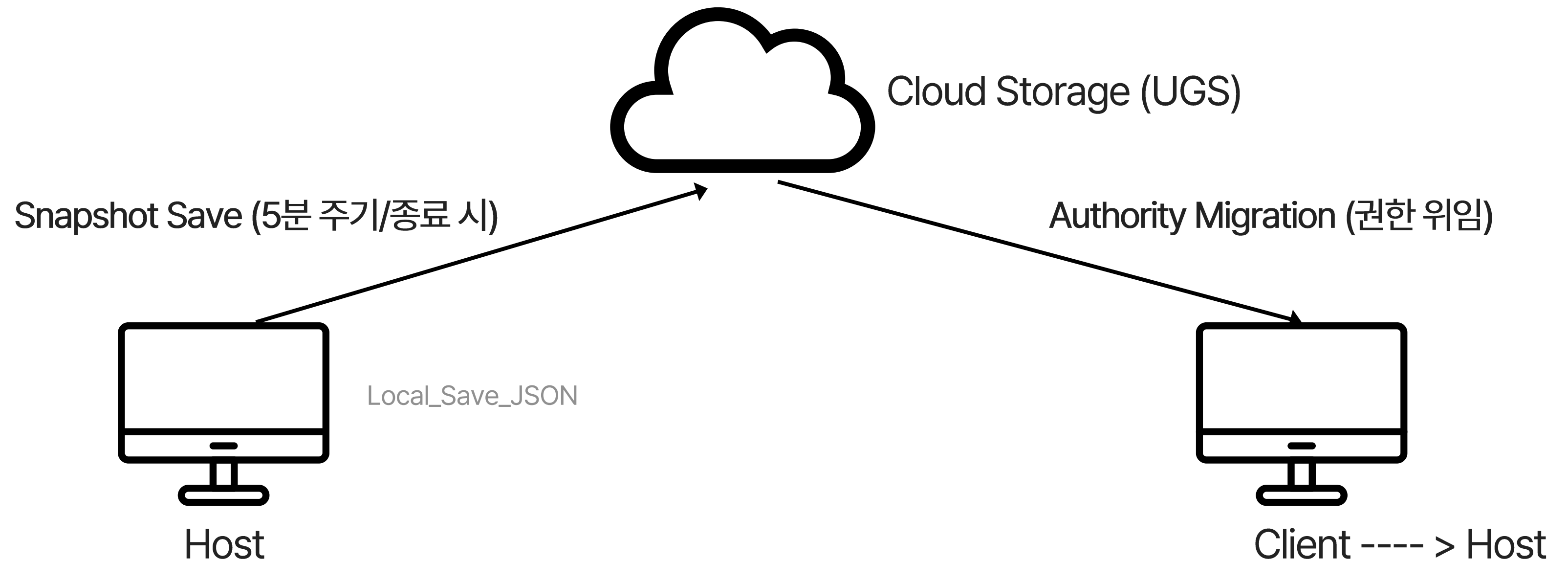
ID	명칭	타입	매입가(NPC→유저)	권장가(유저→NPC)	MBTI 특이점
101	거친 원목	Raw	10 G	15 G	실용적(S) 성향 선호
205	참나무 의자	Furniture	-	120 G	가치 중심(N) 성향 선호

실제 아이템 데이터 예시

I 시스템 데이터 관계도 (ERD)

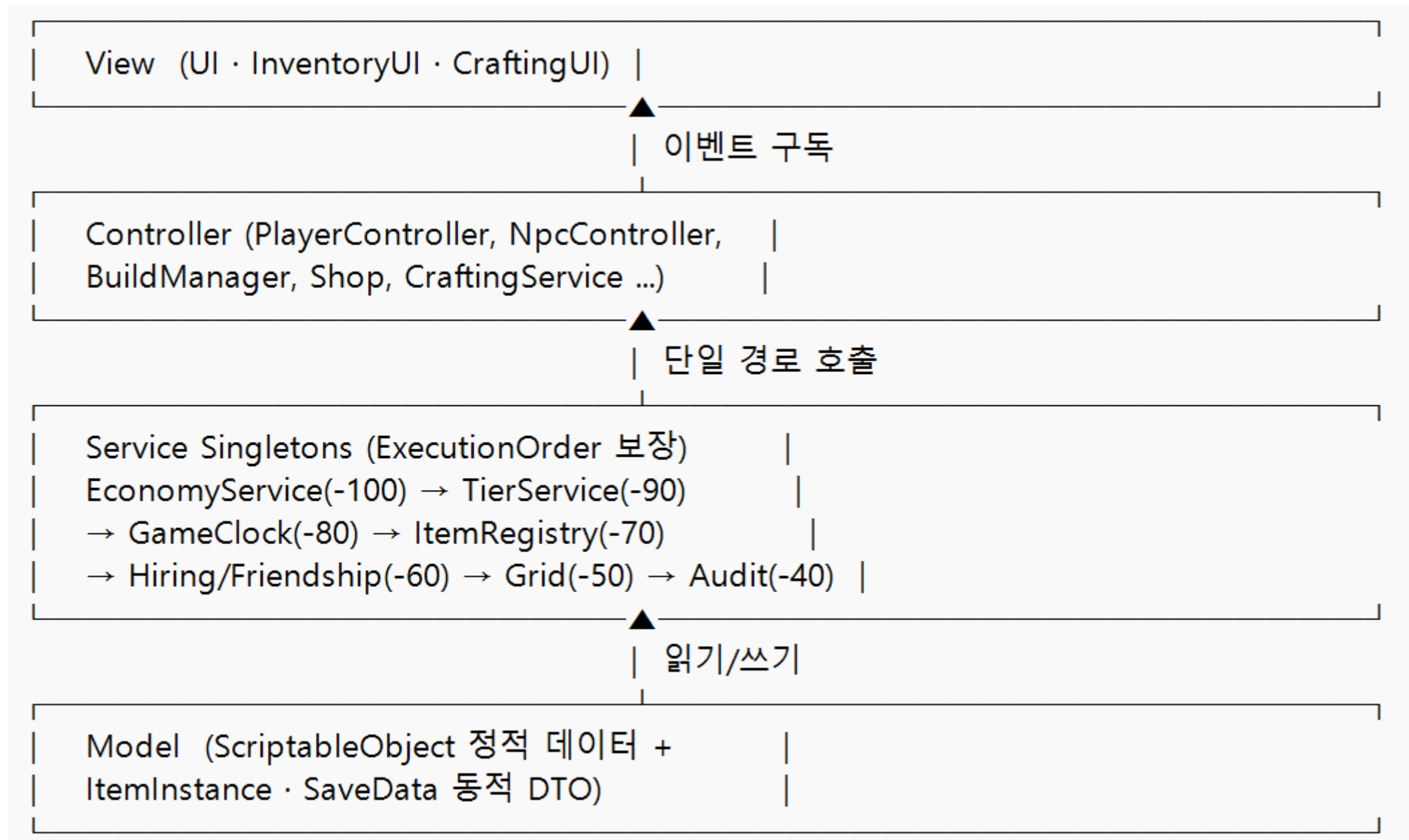


| 멀티플레이 지속성 및 권한 위임

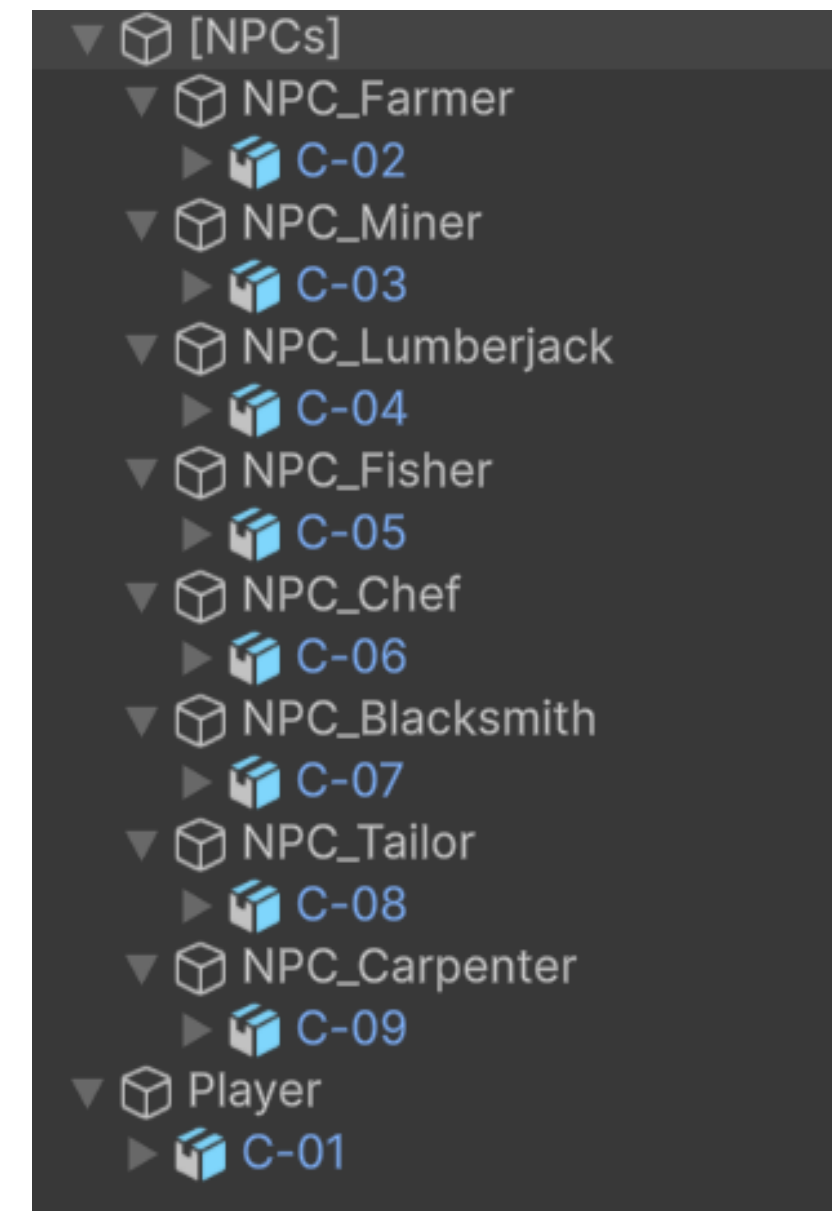
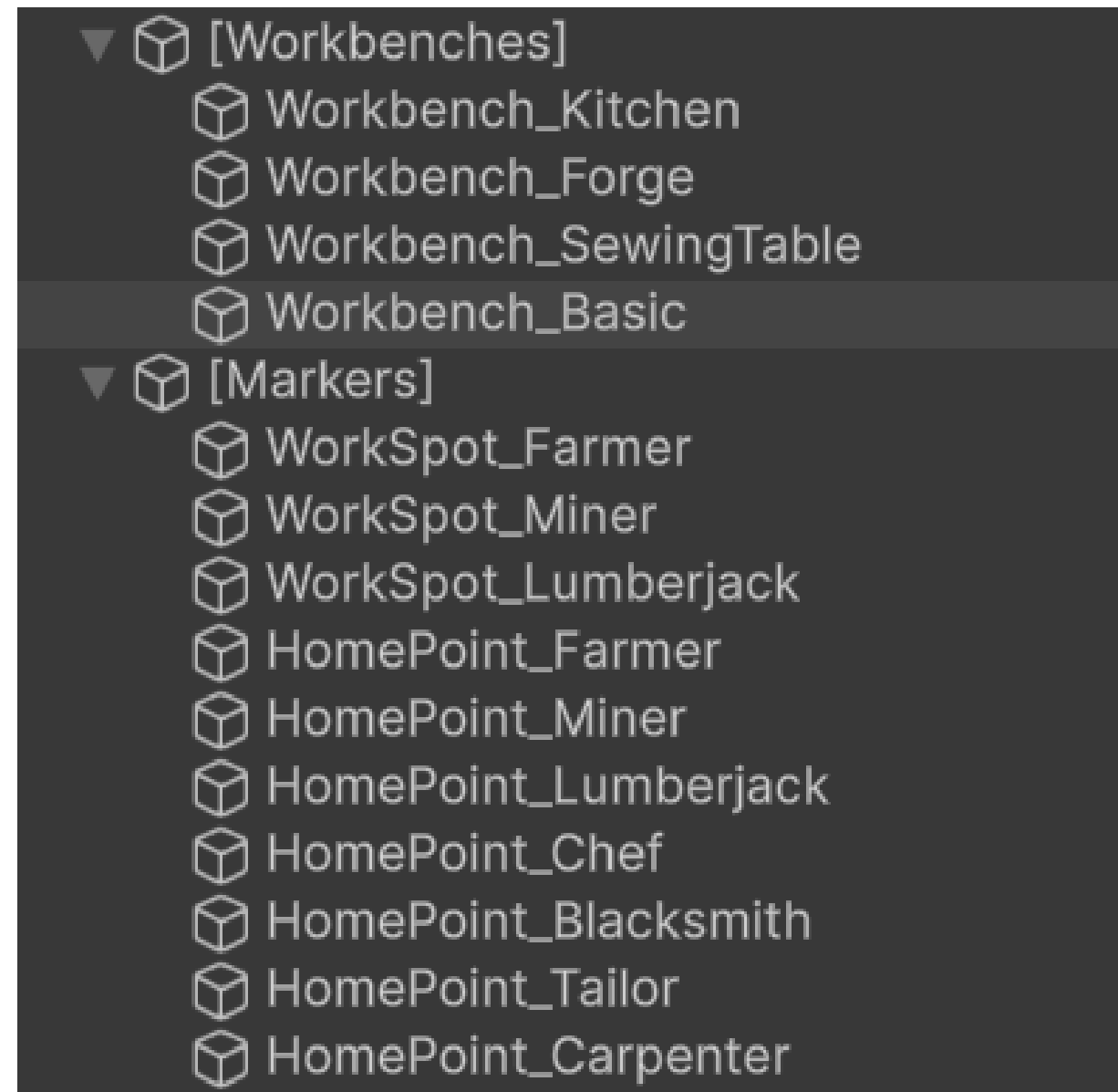
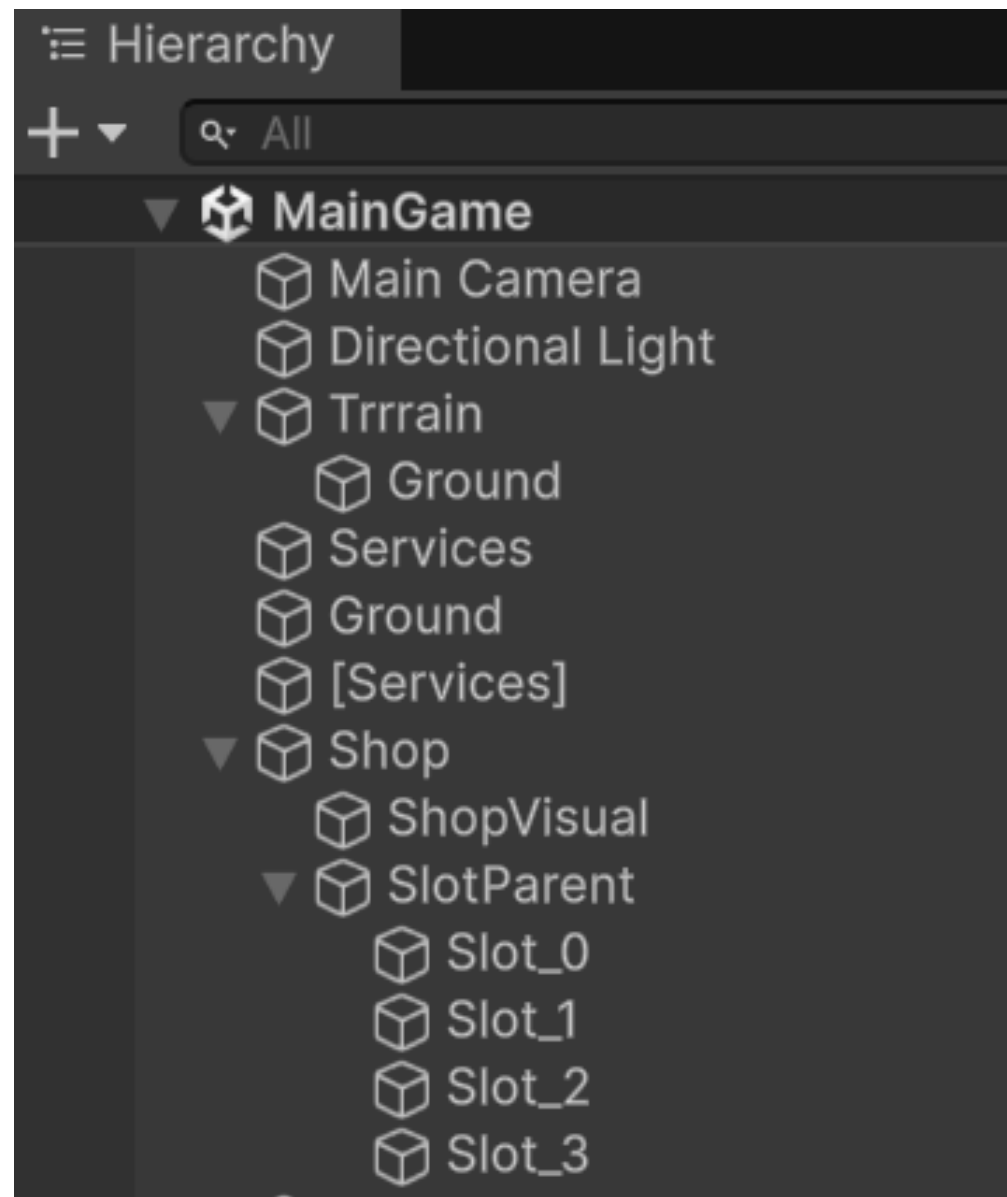


개발 현황

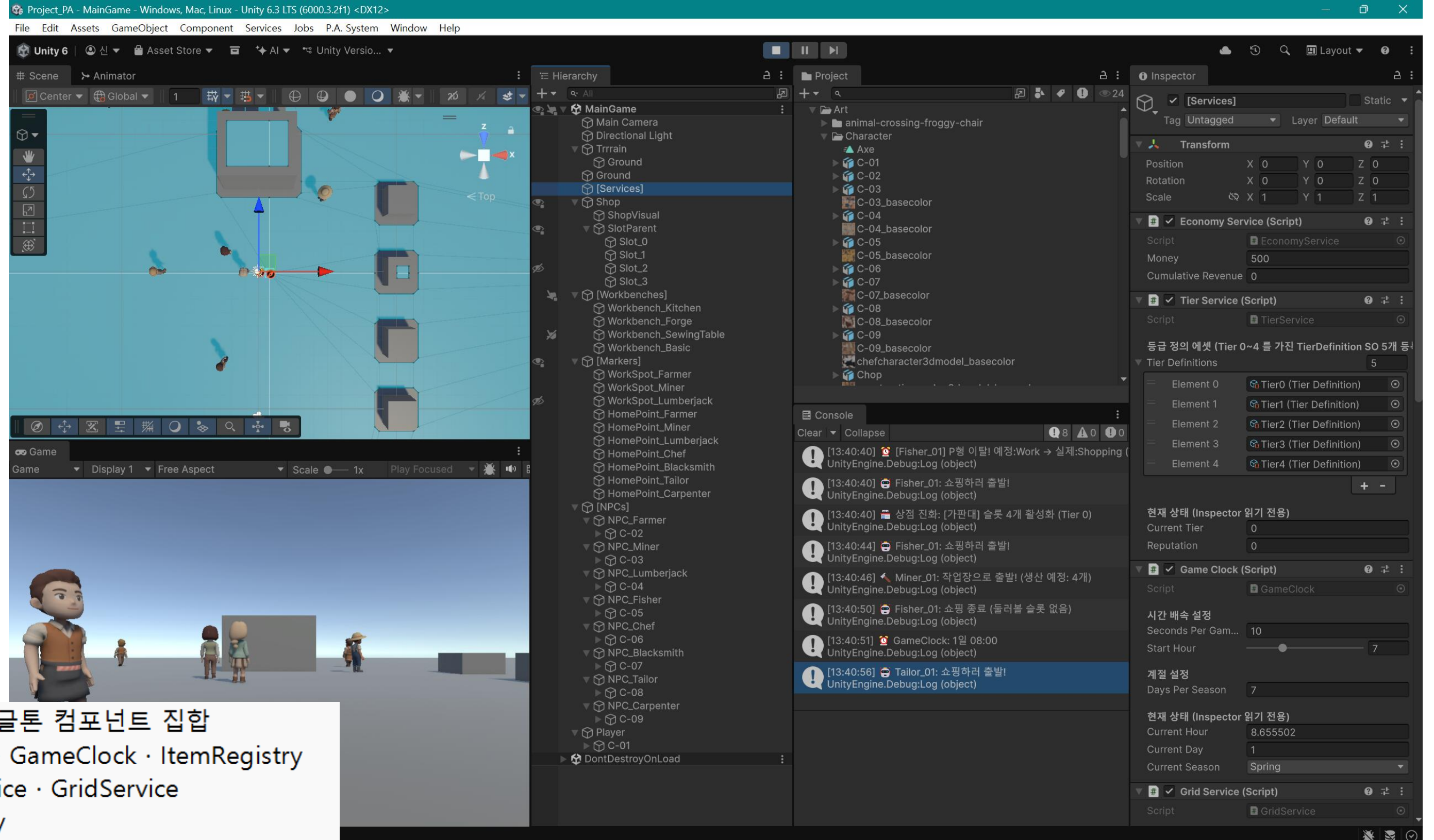
| 아키텍처 개요 Unity식 MVC + 서비스 레이어



| 씬 구성



시스템 구성



[Services] ← 9개 싱글톤 컴포넌트 집합
EconomyService · TierService · GameClock · ItemRegistry
HiringService · FriendshipService · GridService
AuditService · BuildingRegistry

| 모델링 9캐릭터 · 12건물 · 8환경

▶ 캐릭터 9종

C-01 플레이어 (상점 관리자)

C-02~09 NPC 8종

- 생산형 : Farmer · Lumberjack · Miner
- 전문가 : Chef · Blacksmith · Tailor · Carpenter
- 소비형 : 일반 주민 (MBTI 프로필로 개성 주입)



| 모델링 9캐릭터 · 12건물 · 8환경

▶ 건물 12종

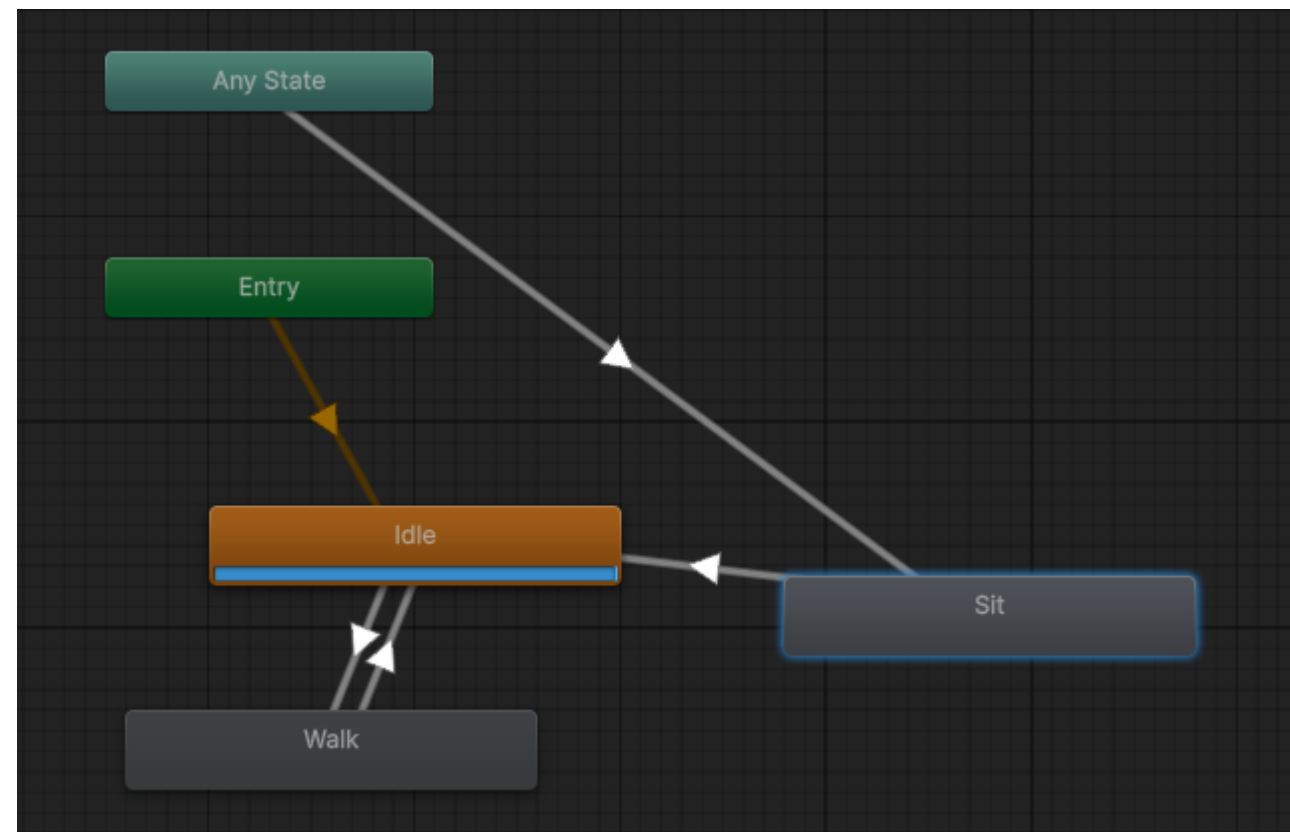
상점 4단계 : 가판대 → 잡화점 → 마트 → 백화점

가공 작업대 : BasicWorkbench · Kitchen · Forge · SewingTable

기타 : 창고 · 농경지 · 플레이어 하우스 · NPC 하우스

▶ 환경 8종 · 상호작용 소품 6종

나무 · 광석 · 수풀 채집 오브젝트, 의자(앉기), 저장 상자 등



▶ 애니메이터 설계

PlayerAnimator.controller

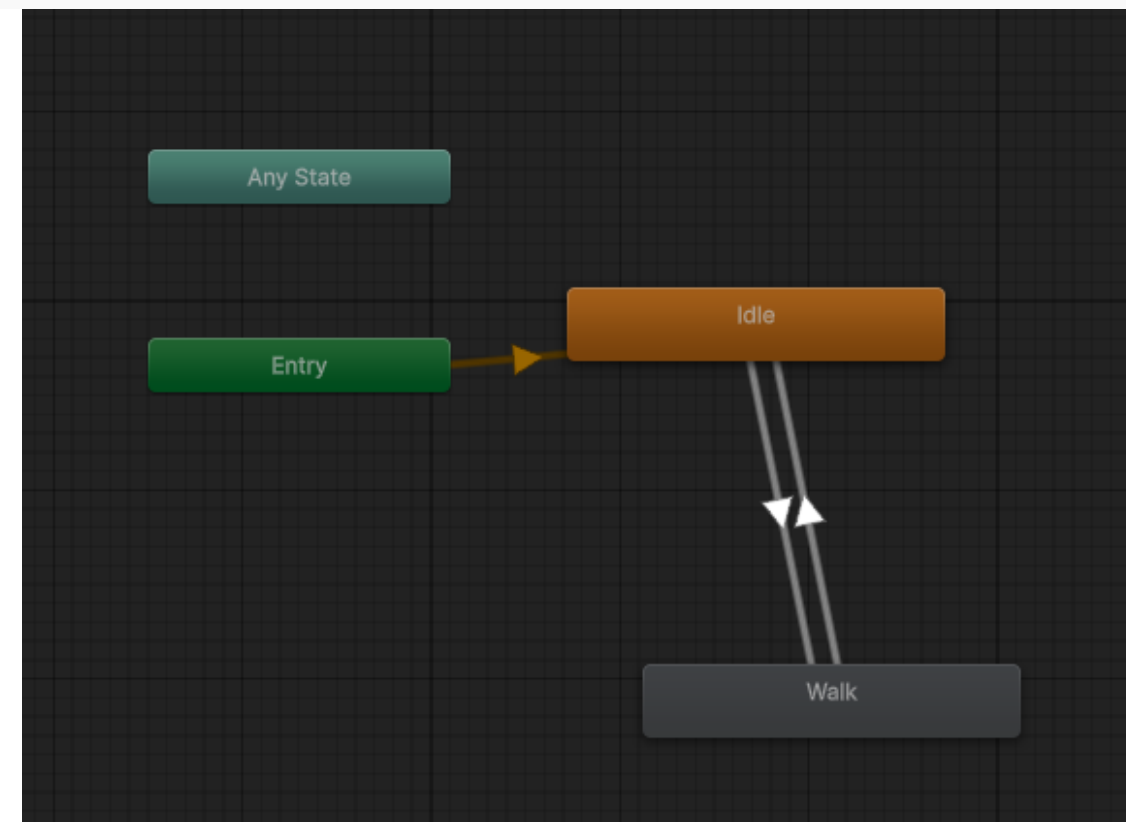
Parameters : Speed (Float), IsSitting (Bool)

States : Idle ↔ Walk (Speed 기반 Blend) + Sit (AnyState)

NpcAnimator.controller

Parameters : Speed (Float)

States : Idle ↔ Walk (NavMeshAgent.velocity.magnitude 연동)



| 코드 아키텍처 레이어

- ▶ Core PlayerController · PlayerInteraction ·
 PlayerInputHandler · CameraController · GameManager
- ▶ Inventory Item · ItemInstance · ItemRegistry · Inventory ·
 InventorySlot · Hotbar · (UI 3종)
- ▶ Build / Grid BuildManager · BuildingData · BuildingRegistry ·
 GridService
- ▶ Shop / Economy Shop · ShopSlot · EconomyService · PurchaseEvaluator
- ▶ NPC AI NpcProfile · NpcController · ProducerNpcController ·
 SpecialistNpcController · NpcScheduleController ·
 NpcDailySchedule · NpcDialogue
- ▶ Hiring / Bond NpcSpecialty · NpcCandidateData · HiringService ·
 HiddenBlueprintData · FriendshipService
- ▶ Dialogue DialogueData · DialogueService
- ▶ Crafting Workbench · RecipeData · CraftingService · CraftingUI
- ▶ Production ProductionData · SeasonModifier
- ▶ Tier / Time TierDefinition · TierService · AuditService · GameClock
- ▶ Save SaveData · SaveManager · ISaveRepository ·
 LocalJsonSaveRepository
- ▶ Farming / Misc Farmland · Crop · Gatherable · StorageBox ·
 EquipmentSystem · Chair · IInteractable

코드 EconomyService

```
1 using System;
2 using UnityEngine;
3
4 // 프로젝트 전체의 '공유 지갑'을 소유하고 모든 돈 변경의 단일 경로가 되는 서비스.
5 //
6 // 설계 의도:
7 // - 멀티플레이 전환 시 "스타듀밸리형 공유 지갑"을 그대로 유지한다.
8 // - 현재는 단일 인스턴스가 int 를 들고 있지만,
9 // - 나중에 내부 _money 를 NetworkVariable<int> 로만 교체하면 호스트 권한 공유 지갑이 된다.
10 // - 모든 변경은 TryModifyMoney 한 경로로만 발생한다.
11 // - 나중에 이 함수만 [ServerRpc] 로 감싸면 치팅 방지 + 권한 검증이 완료된다.
12 // - OnMoneyChanged 이벤트로 UI/시스템이 구독한다 (Observer 패턴, Docs/04 원칙).
13 // - _cumulativeRevenue 는 입금(Deposit)만 누적한다 - TierService 의 승급 조건 기준.
14 [DefaultExecutionOrder(-100)]
15 참조 1개
16 public class EconomyService : MonoBehaviour
17 {
18     참조 3개
19     public static EconomyService Instance { get; private set; }
20
21     참조 10개
22     [SerializeField] private int _money = 0;
23
24     // 누적 총매출 - Deposit 호출 시만 증가한다. TierService 의 승급 조건에 사용.
25     참조 6개
26     [SerializeField] private long _cumulativeRevenue = 0;
27
28     // 현재 잔액(읽기 전용).
29     참조 0개
30     public int Money => _money;
31
32     // 지금까지 벌어들인 누적 총매출(읽기 전용).
33     참조 0개
34     public long CumulativeRevenue => _cumulativeRevenue;
35
36     // 잔액이 변경될 때마다 호출된다. 파라미터: 변경 후 잔액.
37     참조 3개
38     public event Action<int> OnMoneyChanged;
```

```
69
70 // 지출 편의 함수 - 양수를 받아 내부적으로 음수 delta로 변환.
71 참조 0개
72 public bool TrySpend(int amount, string reason)
73 {
74     if (amount < 0) return false;
75     return TryModifyMoney(-amount, reason);
76 }
77
78 // 수입 편의 함수 - 양수 입력 전용. 누적 매출에도 반영된다.
79 참조 0개
80 public bool Deposit(int amount, string reason)
81 {
82     if (amount < 0) return false;
83     if (!TryModifyMoney(amount, reason)) return false;
84
85     // 누적 매출 기록 - Deposit 경로만 누적한다 (지출은 포함하지 않음).
86     _cumulativeRevenue += amount;
87     OnCumulativeRevenueChanged?.Invoke(_cumulativeRevenue);
88     return true;
89 }
90
91 // 저장/로드에서만 쓰는 강제 세팅. 일반 게임플레이 코드가 호출해서는 안 된다.
92 // (MVP 단계에서는 public이지만, 멀티 전환 시 [ServerOnly] 권한이 부여될 자리)
93 참조 0개
94 public void ForceSet(int value, string reason)
95 {
96     _money = Mathf.Max(0, value);
97     Debug.Log($"💰 EconomyService[{reason}]: 강제 세팅 → {_money}G");
98     OnMoneyChanged?.Invoke(_money);
99 }
100
101 // 누적 매출 복구 전용 - 저장/로드 이외 용도 사용 금지.
102 참조 0개
103 public void ForceSetCumulativeRevenue(long value, string reason)
104 {
105     _cumulativeRevenue = Mathf.Max(0L, value);
106     Debug.Log($"💰 EconomyService[{reason}]: 누적 매출 복구 → {_cumulativeRevenue}G");
107     OnCumulativeRevenueChanged?.Invoke(_cumulativeRevenue);
108 }
109 }
```

```
33 // 누적 매출이 변경될 때마다 호출된다. TierService 가 구독하여 승급 조건을 평가한다.
34 참조 2개
35 public event Action<long> OnCumulativeRevenueChanged;
36
37 참조 0개
38 private void Awake()
39 {
40     if (Instance != null && Instance != this)
41     {
42         Destroy(gameObject);
43         return;
44     }
45     Instance = this;
46
47     참조 0개
48     private void Start()
49     {
50         // 초기 구독자들에게 현재 잔액을 브로드캐스트.
51         OnMoneyChanged?.Invoke(_money);
52     }
53
54     // 모든 돈 변경의 단일 진입점.
55     // 음수 delta가 잔액을 0 밑으로 만들면 실패를 반환하고 값을 변경하지 않는다.
56     // reason은 디버깅/감사 로그 용도.
57     참조 2개
58     public bool TryModifyMoney(int delta, string reason)
59     {
60         int next = _money + delta;
61         if (next < 0)
62         {
63             Debug.LogWarning($"🚫 EconomyService: 잔액 부족 ({reason}): 요청 {delta}, 현재 {_money}");
64             return false;
65         }
66
67         _money = next;
68         Debug.Log($"💰 EconomyService[{reason}]: {delta:+#;-#;0}G → {_money}G");
69         OnMoneyChanged?.Invoke(_money);
70         return true;
71     }
72 }
```


코드 NPCProfile

Assets > Scripts > NpcProfile.cs > ...

```
1 using UnityEngine;
2
3 [CreateAssetMenu(fileName = "New NPC Profile", menuName = "P.A. System/NPC Profile")]
4 public class NpcProfile : ScriptableObject
5 {
6     [Header("기본 정보")]
7     public string npcName = "Unnamed";
8     [TextArea] public string bio;
9
10    [Header("MBTI 4축 (-1 ~ +1)")]
11    [Tooltip("-1 = 완전 I, +1 = 완전 E. 광장 체류/쇼핑 확률에 반영")]
12    [Range(-1f, 1f)] public float traitEI = 0f;
13
14    [Tooltip("-1 = 감각(S, 실용재 선호), +1 = 직관(N, 사치품 선호)")]
15    [Range(-1f, 1f)] public float traitSN = 0f;
16
17    [Tooltip("-1 = 사고(T, 논리적 대사), +1 = 감정(F, 감성적 대사). MVP에서는 대사 톤에만 영향")]
18    [Range(-1f, 1f)] public float traitTF = 0f;
19
20    [Tooltip("-1 = 판단(J, 일과 엄격), +1 = 인식(P, 돌발 행동). MVP에서는 FSM drift 에만 영향")]
21    [Range(-1f, 1f)] public float traitJP = 0f;
22
```

Assets > Scripts > NpcProfile.cs > ...

```
4 public class NpcProfile : ScriptableObject
5 {
6     [Range(-1f, 1f)] public float traitJP = 0f;
7
8     [Header("파생 행동 가중치")]
9     [Tooltip("작업 상태에 있을 때의 생산 효율 계수. 1.0 = 기본")]
10    public float workEfficiency = 1f;
11
12    [Tooltip("광장/사교 공간을 선호하는 정도. 0~1")]
13    [Range(0f, 1f)] public float socialWeight = 0.5f;
14
15    [Tooltip("주거지 체류 시간 비율. 0~1 (1 = 거의 집에만)")]
16    [Range(0f, 1f)] public float homeStayBias = 0.3f;
17
18    [Header("소비 가중치 (Docs/03 2.1절)")]
19    [Tooltip("실용재(도구/식재료) 선호 가중치. S 성향이 높이는 계수")]
20    public float utilityConsumption = 1f;
21
22    [Tooltip("사치품(가구/장식) 선호 가중치. N 성향이 높이는 계수")]
23    public float luxuryConsumption = 1f;
24
25    [Tooltip("가격 변화에 대한 저항(민감도). 높을수록 비싸면 금방 구매 포기")]
26    public float priceSensitivity = 1f;
27 }
```

감사합니다